

Maritime CargoService: Sprint 1

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

July 10, 2026

1. Introduction

Il punto di partenza di questo sprint è l'architettura logica (recuperabile al seguente [link](#)^o) e la formalizzazione dei requisiti (recuperabile al seguente [link](#)^o).

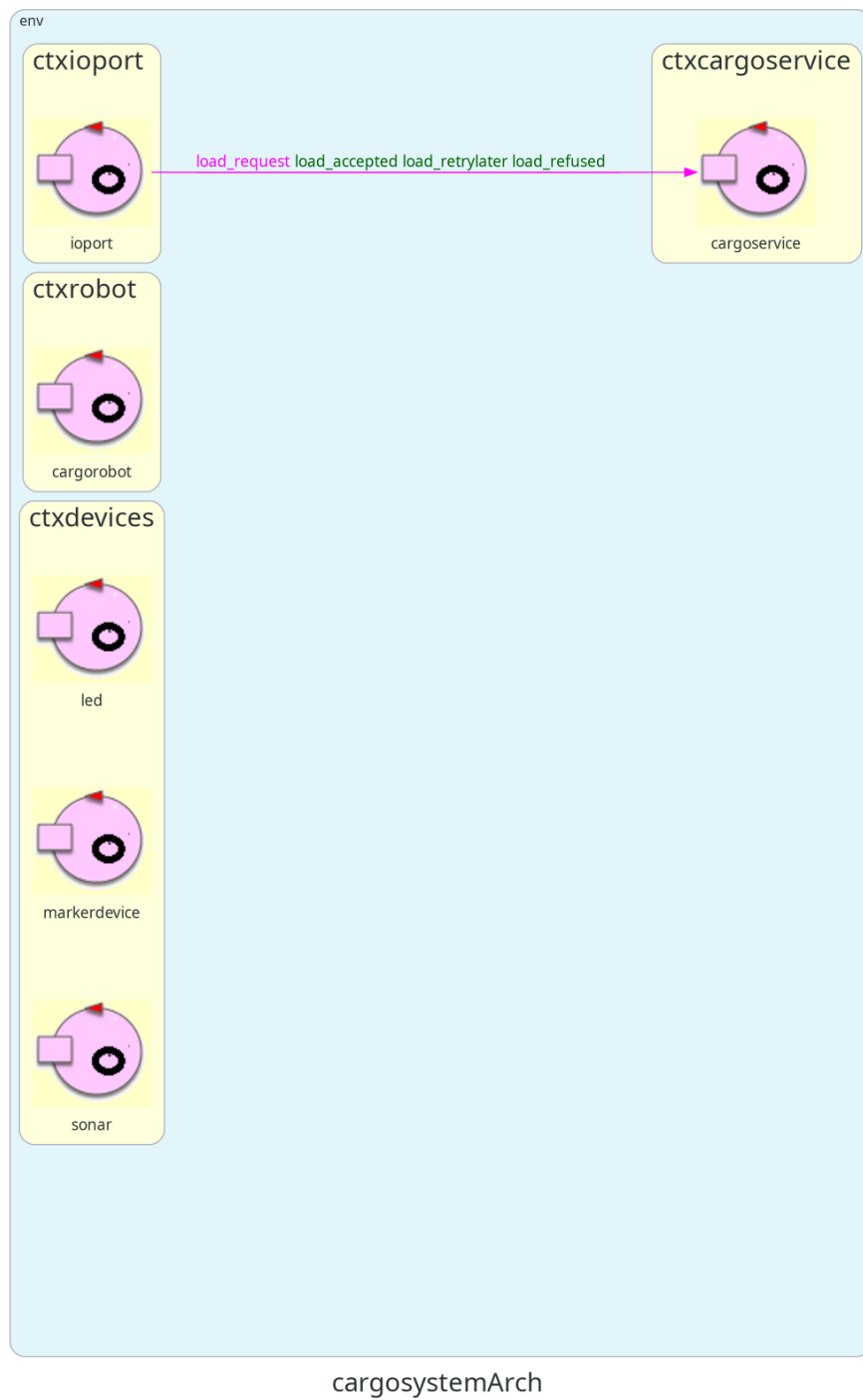


Figure 1: Architettura definita nello Sprint0.

Si riporta di seguito il goal dello Sprint 1.

L'obiettivo dello Sprint 1 è realizzare un prototipo eseguibile del **cargoservice** che implementi il ciclo principale di carico di un container, dalla ricezione della `load_request` fino al deposito del container nello slot riservato. Si prevede di:

- realizzare i componenti non ancora pronti in forma simulata (come mock)
- realizzare il movimento del cargorobot, delegando questo compito ad un componente esterno fornito dalla nostra casa di produzione.

2. Requirements

I requisiti del progetto sono riportati nel documento disponibile al seguente [link](#)^o

L'azienda richiede di realizzare un servizio denominato **cargoservice** con il seguente funzionamento:

- Gli slot1-4 rappresentano le aree della hold riservate per immagazzinare ciascuno un container.
- Lo slot5 rappresenta un'area in cui il cargorobot deve temporaneamente depositare un container, prima di posizionarlo in uno degli slot1-4. Durante la sosta temporanea, un dispositivo 'marker' etichetta il container con un codice a barre identificativo e segnala quando l'attività di marcatura è completata.
- L'IOPort è un dispositivo dotato di un pulsante e di un display. Il pulsante viene premuto dal cliente per inviare una richiesta di carico di un container sulla nave. Il display viene utilizzato per mostrare la risposta alla richiesta e lo stato attuale della hold.
- Il sensore associato all'IOPort è un dispositivo (un sonar) utilizzato per rilevare la presenza di un container, quando misura una distanza D , tale che $D < D_{\text{FREE}}/2$, per un tempo ragionevole (ad esempio 3 secondi).
- Il cargoservice è in grado di ricevere una richiesta di carico di un container inviata da un cliente tramite il pulsante dell'IOPort.
- Invia la risposta **retrylater** se l'IOPort è attualmente occupato da un container oppure se il sistema è **Out of service**.
- Rifiuta la richiesta quando la hold è già piena, ovvero gli slot1-4 sono già tutti occupati.
- Altrimenti, considera il sistema come **engaged**, rileva uno slot libero e restituisce come risposta il nome dello slot riservato. Mentre il sistema è engaged, il LED deve lampeggiare.
- Quando la richiesta di carico viene accettata, il cliente deve spostare il container nell'area del sensore entro un tempo prefissato (ad esempio 30 secondi), altrimenti il sistema diventa **disengaged**.
- Successivamente, il cargoservice utilizza il cargorobot per spostare il container dall'IOPort a slot5 (per l'etichettatura del container) e poi allo slot riservato.

- Il servizio deve inoltre mostrare sul display dell'IOPort:
 - lo stato attuale della hold
 - il messaggio “**Service working**” quando tutto sta procedendo correttamente
 - il messaggio “**Out of service**” se il sensore sonar misura la distanza (del container dal sonar stesso) $D > D_{\text{FREE}}$ per almeno 3 secondi (possibile guasto del sonar)

3. Requirement analysis

Una analisi approfondita dei requisiti è stata svolta nello [Sprint0°](#). Tali risultati vengono assunti come validi anche per lo Sprint 1.

Nel corso dello Sprint 1 è stato tuttavia effettuato un ulteriore approfondimento dei requisiti, a seguito di un chiarimento richiesto alla committente.

Domanda al committente.

Comportamento del robot: Cosa succede se il sistema entra nello stato di out-of-service? Cosa succede al robot una volta che il sistema esce da tale stato?

Risposta della committente Il robot deve fermarsi e, una volta terminato lo stato di out-of-service, deve riprendere il movimento se originariamente era in esecuzione.

Da tale chiarimento si deduce che il **cargorobot** debba essere modellato come un servizio in quanto costituisce una componente reattiva del sistema. Non è invece specificato quale componente debba rilevare la condizione di guasto; tale aspetto viene pertanto rimandato alla successiva analisi del problema.

4. Problem analysis

L'implementazione completa del sistema presuppone dell'esistenza di altri componenti del sistema fisici o in forma simulata (**sonar**, **LED**, **markerdevice**, **IOPort** come web GUI) non ancora sviluppati. La loro realizzazione concreta è pianificata per gli sprint successivi. I componenti “mock” che replicheranno il comportamento di quelli mancanti verranno evoluti rispetto alla formalizzazione QAK già avvenuta in fase di Sprint0 (recuperabile al seguente [link°](#)).

Come emerso dall'analisi dei requisiti, questo componente funge da **orchestratore**: coordina le operazioni degli altri componenti del sistema al fine di eseguire le procedure di carico. Inoltre, le entità sono distribuite su quattro nodi separati ma, per non rallentare la prototipazione, tutti i componenti verranno rappresentati nello stesso nodo (rappresentato da un Context), tenendo in considerazione che gli attori non condividono memoria, e comunicano tra loro tramite scambio di messaggi.

4.1. Rappresentazione dello stato interno della hold

In questa fase la gestione della hold viene rappresentata tramite un **POJO**.

La scelta è motivata dal fatto che la hold non rappresenta un'entità attiva del sistema, ma una struttura dati (già definita in fase di sprint0) che descrive lo stato interno

dell'ambiente: posizione degli slot, ostacoli, IOPort, home del robot e stato di occupazione degli slot.

L'interfaccia `IHold` definisce le operazioni necessarie per la gestione della hold, indipendentemente dalla loro implementazione. La classe `Hold` ne costituisce l'implementazione concreta, occupandosi della rappresentazione del suo stato e della logica di assegnazione degli slot.

Inoltre, lo stato iniziale della hold viene caricato da un file di configurazione JSON, così da evitare valori hard-coded nel codice e rendere più semplice modificare la disposizione dell'ambiente senza cambiare la logica applicativa.

Il file di configurazione è disponibile al seguente link: [configurazione hold](#)°

Il codice della classe `Hold` è disponibile al seguente link: [codice Hold](#)°

Il codice dell'interfaccia `IHold` è disponibile al seguente link: [codice Hold](#)°

```
public interface IHold {
    int doReserveSlot();
    void doFreeSlot(int slotId);
    int doGetSlotX(int slotId);
    int doGetSlotY(int slotId);
    int doGetHomeX();
    int doGetHomeY();
    CellType getCell(int x, int y);
    boolean isOccupied(int slotId);
}
```

4.2. Analisi e scelta del componente cargorobot

La realizzazione del **cargorobot** richiede di stabilire la natura software più adeguata per il componente. A nostra disposizione abbiamo quattro componenti già pronti, forniti dalla casa di produzione, per la modellazione di un Differential Drive Robot (DDR):

- **VirtualRobot26**: È un ambiente virtuale che non fornisce un'interfaccia di programmazione diretta, ma interagisce ricevendo comandi di movimento

[Documentazione VirtualRobot26](#)° [Codice VirtualRobot26](#)°

- **RobotObj26**: È un POJO. Essendo un oggetto passivo, è limitato e può essere utilizzato solo se il chiamante è scritto in Java.
 - **Pro**: Fornisce comandi chiari e semplici da invocare via codice.
 - **Contro**: Approccio passivo, sincrono e non reattivo; vincolato al linguaggio Java e inadatto per architetture distribuite o a microservizi.

[Documentazione RobotObj26](#)° [Codice RobotObj26](#)°

- **RobotService26**: È un servizio reattivo che richiede che l'applicazione chiamante gestisca manualmente la logica di navigazione passo dopo passo.
 - **Pro**: Architettura reattiva e orientata allo scambio di messaggi.

- **Contro:** Obbliga il chiamante a farsi carico di tutta la logica di routing e controllo ostacoli.

[Documentazione RobotService26°](#) [Codice RobotService26°](#)

- **RobotSmart26:** È un servizio avanzato, proattivo e reattivo. Strutturato su tre servizi (`robotmnemo`, `planexec`, `robotsmart`), è capace di ricevere coordinate, pianificare autonomamente il percorso tramite algoritmo A* ed elaborare interruzioni impreviste.
 - **Pro:** Totale autonomia di navigazione, consapevolezza dello spazio (grazie alla mappa interna) e gestione autonoma delle interruzioni.
 - **Contro:** Maggiore complessità architetturale interna dovuta al necessario coordinamento tra più attori.

[Documentazione RobotSmart26°](#) [Codice RobotSmart26°](#)

Per scegliere se vedere al **cargorobot** come un POJO o come un Servizio, il fattore principale è la **reattività**. Da requisito, in caso di anomalia al sonar (**Out of service**) il robot deve essere in grado di fermarsi completamente. Questa esigenza di reattività e proattività ad eventi di sistema giustifica la modellazione del robot come **Servizio** e non come un semplice **POJO**.

È stato concordato l'utilizzo del servizio **RobotSmart26** per garantire **reattività** in caso di allarmi (es. sonar guasto), poiché esso è progettato per interrompere i piani di movimento e mantenere una coerenza di stato. Inoltre possiede la Request `moverobot(TARGETX, TARGETY)`, che formalizza una richiesta di spostamento, calcolando ed eseguendo automaticamente il piano per raggiungere la cella.

Analizzando il codice di **RobotSmart26**, si osserva che il componente soddisfa i requisiti funzionali richiesti al cargorobot. In particolare, il robot mantiene internamente una rappresentazione della mappa dell'ambiente che coincide con la struttura della hold prevista dal sistema; l'unica informazione non gestita riguarda lo stato di occupazione degli slot, demandato al cargoservice. Per questo motivo, si è scelto di adottare RobotSmart26 come implementazione del cargorobot, riutilizzandone il comportamento senza introdurre modifiche sostanziali.

La documentazione di riferimento per **robotsmart26** è disponibile al seguente link: [robotsmart26°](#)

La specifica Qak del componente **robotsmart26** è disponibile al seguente link: [robotsmart26°](#)

4.3. Analisi delle Interazioni

Di seguito si riporta una formalizzazione delle interazioni tra gli attori modellate interpretando e applicando opportune scelte progettuali sui requisiti del sistema. Il codice è disponibile al seguente [link°](#).

- Da requisiti, sappiamo che il sonar deve misurare continuamente la distanza del container dal sonar stesso. Nasce quindi la necessità di dover trasmettere queste mis-

urazioni al `cargoservice`. Per isolare la responsabilità del sonar a semplice “misuratore e trasmettitore” di informazione, viene naturale formalizzare tale comunicazione come un Event, ovvero un messaggio broadcast che verrà ascoltato da chi interessato (in questo caso **cargoservice**)

```
Event    sonardata          : distance(D)
```

- L’invio di comandi al led è delegato al `cargoservice`. Non è necessario attendere una risposta, quindi utilizziamo una Dispatch.

```
Dispatch led_ctrl          : ledCmd(CMD)
// CMD: "on", "off", "blink"
```

- Il `cargoservice` deve interrogare il `markerdevice` per sapere quando l’operazione di etichettatura è terminata, non potendo procedere finché il `markerdevice` non ha risposto. Usiamo quindi la Request/Reply.

```
// CargoService ↔ MarkerDevice
Request mark_container : markContainer(none)
Reply  marking_done   : markingDone(none) for mark_container
```

- Per consentire lo spostamento del robot da una posizione a un’altra, `cargoservice` invia all’attore `cargorobot` (che funge da wrapper di `robosmart26`) una **request**, attraverso la quale è possibile specificare la destinazione (tramite coordinate) e il tempo di esecuzione. Alla ricezione della richiesta, `cargorobot` calcola il percorso verso la destinazione ed esegue il movimento del robot.

```
Request moverobot : moverobot(TARGETX, TARGETY, STEPTIME)
```

Al termine dell’operazione può rispondere con due possibili messaggi, uno per confermare l’avvenuto completamento del percorso e l’altro per segnalare eventuali problemi durante l’esecuzione del movimento. In quest’ultimo caso, il messaggio di risposta conterrà due parametri che indicano la parte di percorso già eseguita (`PLANDONE`) e quella ancora da percorrere (`PLANTODO`).

```
Reply moverobotdone : moverobotok(ARG) for moverobot
Reply moverobotfailed : moverobotfailed(PLANDONE, PLANTODO) for moverobot
```

4.4. Rappresentazione dell’attore `cargoservice` come Macchina a Stati Finiti

L’attore `cargoservice` è, già da Sprint 0, inteso come una Macchina a Stati Finiti che utilizza variabili interne per:

- memorizzare se la porta di ingresso risulta occupata (`IOPortOccupied`)
- se il servizio è attualmente operativo (`ServiceWorking`)

- lo stato logico del servizio (CargoState)
- l'identificativo dello slot eventualmente riservato (ReservedSlotId).
- durata del passo (in ms) di avanzamento di una singola cella da parte del robot (StepTime)

Per quanto riguarda le transizioni della FSM, si estende il comportamento già realizzato in Sprint0.

```
Context ctxprototype ip [host="localhost" port=8050] // Contesto unico del
prototipo per lo Sprint 1
Context ctxrobotsmart      ip [host="127.0.0.1" port=8020]

ExternalQActor robotsmart context ctxrobotsmart

QActor cargoservice context ctxprototype {
    [#
        var IOPortOccupied = false
        var ServiceWorking = true
        var CargoState      = "disengaged"
        var ReservedSlotId  = -1
        val StepTime        = 345
    #]
```

- Quando viene ricevuta una richiesta **load_request**, il cargoservice prima di prenotare uno slot verifica che:
 1. il servizio non sia già impegnato nella gestione di un altro container
 2. il sistema risulti funzionante
 3. che la porta di ingresso sia libera.

Se tutte le condizioni risultino soddisfatte viene interrogata la Hold per richiedere la prenotazione di uno slot libero, il cargoservice memorizza l'identificativo dello slot riservato, passa allo stato engaged e informa il client dell'avvenuta accettazione della richiesta.

Viene inoltre attivato il LED in modalità **blink** e viene comunicato l'ID dello slot (da mostrare in futuro sul display). Nel caso in cui il magazzino risulti completamente occupato cargoservice comunicherà il rifiuto della richiesta, rimanendo disponibile per eventuali.

Il timer di 30 secondi viene avviato quando il sistema entra nello stato **engaged**. Se il container non viene posizionato nell'area del sensore entro tale intervallo, il sistema passa allo stato **disengaged**, libera lo slot precedentemente riservato e spegne il LED.

```
State s0 initial {
    println("cargoservice | AVVIATO") color magenta
}
Goto disengaged

State disengaged {
    println("cargoservice | LIBERO: in attesa di richieste...") color
```

```

blue
}
Transition t0
    whenRequest load_request → handle_load_request
    whenEvent    sonardata    → handle_sonar

    State engaged {
        println("cargoservice | IMPEGNATO: in attesa del deposito del
container...") color blue
    }
    Transition t0
        whenTime    30000      → handle_deposit_timeout
        whenRequest load_request → handle_load_request
        whenEvent    sonardata  → handle_sonar

    State handle_load_request {
        printCurrentMessage color yellow

        // Verifica precondizioni per accettare la richiesta
        if [# CargoState = "engaged" || !ServiceWorking || IOPortOccupied
#] {
            replyTo load_request with load_retrylater : loadRetryLater(none)
        } else {
            // Interroga il POJO Hold per uno slot libero
            [# val SlotId = Hold.reserveSlot() #]
            if [# SlotId > 0 #] {
                [#
                    ReservedSlotId = SlotId
                    CargoState      = "engaged"
                #]
                forward ledmock -m led_ctrl : ledCmd(blink)
                [# val SlotName = "slot$ReservedSlotId" #]
                replyTo load_request with load_accepted :
loadAccepted($SlotName)
            } else {
                replyTo load_request with load_refused : loadRefused(none)
            }
        }
    }
    }
    Goto engaged if [# CargoState = "engaged" #] else disengaged

```

- Il cargoservic interpreta gli eventi emessi dal sonar in tre modi principali:
 1. Viene rilevata la presenza di un container entro una certa distanza dal sensore (per almeno 3s) e viene impostata a true la variabile interna **IOPortOccupied** per indicare che l'IOPort è occupata.
 2. Viene rilevata una distanza superiore a DFREE (per almeno 3s). Il sistema viene in questo caso messo nello stato **Out of service**. Viene quindi aggiornata la variabile interna **ServiceWorking**.
 3. La distanza misurata non implica nessun cambiamento di stato. In questo caso il sistema continua a funzionare normalmente.

Al termine dell'elaborazione dell'evento il **cargoservice** valuta se siano contemporaneamente soddisfatte tutte le condizioni necessarie per iniziare la movimentazione del container. Il robot viene attivato solamente quando il servizio si trova nello stato engaged, il container è stato effettivamente depositato e il sistema risulta operativo.

Per semplicità, nello Sprint 1 non viene implementata la verifica della persistenza della misura per almeno 3 secondi richiesta dalla traccia. Il prototipo assume che ogni evento sonardata rappresenti una misura già validata dal sonar. La gestione completa del vincolo temporale verrà introdotta negli sprint successivi.

```

State handle_sonar {
  onMsg(sonardata : distance(D)) {
    [# val Dist = payloadArg(0).toInt() #]

    // D < 50 indica presenza del container (IOPort occupata)
    if [# Dist < 50 #] {
      [# IOPortOccupied = true #]
    } else {
      [# IOPortOccupied = false #]
    }

    // D > DFREE (es. > 150) per un intervallo prolungato indica
    condizione OUT OF SERVICE
    if [# Dist > 150 #] {
      [# ServiceWorking = false #]
      println("cargoservice | Sonar D > DFREE ($Dist) → Sistema
FUORI SERVIZIO!") color red
    } else {
      if [# !ServiceWorking #] {
        println("cargoservice | Sonar D ≤ DFREE ($Dist) →
Sistema di nuovo FUNZIONANTE!") color green
      }
      [# ServiceWorking = true #]
    }
  }
}

// Se il container viene posato durante lo stato engaged e il sistema è
operativo, il robot può iniziare
Goto do_robot_job if [# IOPortOccupied && CargoState = "engaged" &&
ServiceWorking #] else returnToState

State returnToState {}
Goto engaged if [# CargoState = "engaged" #] else disengaged

```

La procedura di deposito è centralizzata nel **cargoservice**, mentre le operazioni di movimentazione del robot e di marcatura sono modellate tramite comunicazioni Request/Reply. In questo modo è garantita una sincronizzazione esplicita tra le fasi della procedura, impedendo che il deposito nello slot definitivo avvenga prima del completamento della marcatura.

Le coordinate delle destinazioni, invece, vengono recuperate dalla Hold. Affinchè ogni nuova operazione inizi da una configurazione nota, al termine della procedura il robot

viene riportato automaticamente nella posizione **Home**. Per consentire una successiva gestione del guasto, in caso di fallimento di uno spostamento, il cargoservice mantiene il sistema nello stato engaged e conserva la prenotazione dello slot. Si è scelto di non annullare automaticamente la procedura.

```

State do_robot_job {
    println("cargoservice | Container depositato! Spostamento
del robot allo slot5 (2,5) [Riga,Colonna] tramite cargorobot (wrapper
robotsmart26)...") color magenta
    request cargorobot -m moverobot : moverobot(2, 5, $StepTime)
}
Transition t0
    whenReply moverobotdone → mark_container
    whenReply moverobotfailed → handle_robot_fail

State mark_container {
    println("cargoservice | Allo slot5. Richiesta di marcatura al
markerdevice...") color magenta
    request markerdevice -m mark_container : markContainer(none)
}
Transition t0
    whenReply marking_done → move_to_reserved_slot

State move_to_reserved_slot {
    [#
        val DestX = Hold.getSlotX(ReservedSlotId)
        val DestY = Hold.getSlotY(ReservedSlotId)
    #]
    println("cargoservice | Marcato! Spostamento del container allo
slot$ReservedSlotId ($DestY, $DestX) [Riga,Colonna] tramite cargorobot...")
color magenta
    request cargorobot -m moverobot : moverobot($DestY, $DestX,
$StepTime)
}
Transition t0
    whenReply moverobotdone → return_home
    whenReply moverobotfailed → handle_robot_fail

State return_home {
    [#
        val HomeX = Hold.getHomeX()
        val HomeY = Hold.getHomeY()
    #]
    println("cargoservice | Container immagazzinato! Ritorno del
robot alla HOME ($HomeY,$HomeX) [Riga,Colonna] tramite cargorobot...") color
magenta
    request cargorobot -m moverobot : moverobot($HomeY, $HomeX,
$StepTime)
}
Transition t0
    whenReply moverobotdone → finish_job
    whenReply moverobotfailed → handle_robot_fail

State finish_job {

```

```

        println("cargoservice | Lavoro completato!") color green
        [# CargoState = "disengaged" #]
        forward ledmock -m led_ctrl : ledCmd(off)
    }
    Goto disengaged

    State handle_robot_fail {
        println("cargoservice | Movimento del robot fallito!") color red
    }
    Goto engaged

    State handle_deposit_timeout {
        println("cargoservice | Timeout del deposito! Liberazione dello
slot.") color red

        [#
            CargoState = "disengaged"
            Hold.freeSlot(ReservedSlotId)
        #]
        forward ledmock -m led_ctrl : ledCmd(off)
    }
    Goto disengaged
}

```

4.5. Attori QAK di supporto

Gli attori QAK che rappresentano i dispositivi simulati sono riportati separando il comportamento di supporto al prototipo dalle parti usate esclusivamente come testplan.

```

QActor cargorobot context ctxprototype {
    State s0 initial {
        println("cargorobot | STARTED (Wrapper di robotsmart26)") color blue
    }
    Goto waiting

    State waiting {}
    Transition t0
        whenRequest moverobot → handle_move

    State handle_move {
        onMsg(moverobot : moverobot(X, Y, TIME)) {
            [#
                val Tx = payloadArg(0)
                val Ty = payloadArg(1)
                val StepTime = payloadArg(2)
            #]
            println("cargorobot | Inoltro comando moverobot($Tx, $Ty,
$StepTime) a robotsmart26...") color cyan
            request robotsmart -m moverobot : moverobot($Tx, $Ty, $StepTime)
        }
    }
    Transition t0
        whenReply moverobotdone → handle_move_done
        whenReply moverobotfailed → handle_move_failed
}

```

```

    State handle_move_done {
        onMsg(moverobotdone : moverobotok(ARG)) {
            println("cargorobot | Movimento completato da robotsmart26.
Rispondo al cargoservice...") color cyan
            replyTo moverobot with moverobotdone : moverobotok(done)
        }
    }
    Goto waiting

    State handle_move_failed {
        onMsg(moverobotfailed : moverobotfailed(PL, TIME)) {
            println("cargorobot | Fallimento movimento in robotsmart26.
Rispondo failure al cargoservice...") color red
            replyTo moverobot with moverobotfailed :
moverobotfailed(obstacle, 0)
        }
    }
    Goto waiting
}

QActor markerdevice context ctxprototype {
    State s0 initial {
        println("markerdevice | AVVIATO") color green
    }
    Goto work

    State work {}
    Transition t0
        whenRequest mark_container → handle_mark

    State handle_mark {
        println("markerdevice | Marcatura del container in corso...") color
cyan
        delay 1500
        println("markerdevice | Container marcato!") color cyan
        replyTo mark_container with marking_done : markingDone(none)
    }
    Goto work
}

QActor ledmock context ctxprototype {
    State s0 initial { }
    Goto work

    State work {}
    Transition t0
        whenMsg led_ctrl → handle_cmd

    State handle_cmd {
        onMsg(led_ctrl : ledCmd(CMD)) {
            println("ledmock | Il LED ora è ${payloadArg(0)}") color green
        }
    }
}

```

```
Goto work
}
```

5. Test plans

Il testplan dello Sprint1 viene realizzato come scenario eseguibile interno al modello QAK (recuperabile al seguente [link](#)^o). Gli attori mock avviano automaticamente le interazioni necessarie a verificare il comportamento essenziale del `cargoservice`.

I test previsti sono:

- **TEST 1: richiesta accettata**

Eseguito da `ioportmock` : invia una prima `load_request` quando il servizio è libero. Risultato atteso: `cargoservice` risponde con `load_accepted(slotN)` e invia `ledCmd(blink)` a `ledmock`.

- **TEST 2: nessun accodamento**

Eseguito da `ioportmock` : invia subito una seconda `load_request` mentre il servizio è `engaged`.

Risultato atteso: `cargoservice` risponde con `load_retrylater`, quindi la richiesta non viene accodata.

```
QActor ioportmock context ctxprototype {
  State s0 initial {
    // TEST 1:
    delay 1000
    println("ioportmock | TEST 1 - Invio della 1a load_request (atteso
load_accepted)") color cyan
    request cargoservice -m load_request : loadRequest(none)
  }
  Transition t0
    whenReply load_accepted → handle_accept
    whenReply load_retrylater → handle_retry
    whenReply load_refused → handle_refuse

  State handle_accept {
    printCurrentMessage color green

    // TEST 2:
    delay 500
    println("ioportmock | TEST 2 - Invio della 2a load_request mentre
impegnato (atteso load_retrylater)") color cyan
    request cargoservice -m load_request : loadRequest(none)
  }
  Transition t0
    whenReply load_retrylater → handle_no_queue_ok
    whenReply load_accepted → handle_unexpected_accept
    whenReply load_refused → handle_refuse

  State handle_no_queue_ok {
    printCurrentMessage color yellow
  }
}
```

```

        println("ioportmock | TEST 2 OK - richiesta rifiutata con retrylater
mentre il servizio è impegnato") color green
    }

    State handle_unexpected_accept {
        printCurrentMessage color red
        println("ioportmock | TEST 2 FALLITO - la richiesta è stata accettata
mentre il servizio era già impegnato") color red
    }
    State handle_retry { printCurrentMessage color yellow }
    State handle_refuse { printCurrentMessage color red }
}

```

- **TEST 3: deposito del container**

Eseguito da `sonarmock` : emette `sonardata: distance(30)` .

Risultato atteso: `cargoservice` rileva l'IOPort occupato e avvia la procedura di movimentazione del robot.

- **TEST 4: sonar out of service**

Eseguito da `sonarmock` : emette `sonardata: distance(200)` .

Risultato atteso: `cargoservice` imposta il sistema in stato **Out of service**.

- **TEST 5: ripristino sonar**

Eseguito da `sonarmock` : emette `sonardata: distance(100)` .

Risultato atteso: `cargoservice` riporta il sistema in stato **Working**.

```

QActor sonarmock context ctxprototype {
    State s0 initial {
        // TEST 3:
        delay 4000
        println("sonarmock | Container posizionato → Emissione
sonardata(30)") color cyan
        emit sonardata : distance(30)

        // TEST 4:
        delay 8000
        println("sonarmock | Il sonar misura D > DFREE (200). Emissione
sonardata(200)") color cyan
        emit sonardata : distance(200)

        // TEST 5:
        delay 5000
        println("sonarmock | Il sonar misura D ≤ DFREE (100). Emissione
sonardata(100)") color green
        emit sonardata : distance(100)
    }
}

```

- **TEST 6: marcatura container**

Eseguito da `markerdevice` : riceve `mark_container` e risponde `marking_done` .

Risultato atteso: `cargoservice` prosegue verso il trasferimento allo slot riservato.

```

QActor markerdevice context ctxprototype {
    State handle_mark {
        // TEST 6:

        println("markerdevice | Marcatura del container in corso...") color
cyan
        delay 1500
        println("markerdevice | Container marcato!") color cyan
        replyTo mark_container with marking_done : markingDone(none)
    }
}

```

L'esecuzione del testplan avviene avviando il prototipo:

```

cd sprint1/prototype
./gradlew run

```

Le evidenze dei test sono visibili direttamente nel log prodotto dagli attori mock e dal `cargoservice`.

6. Deployment

Per procedere al deployment del prototipo è necessario eseguire i seguenti passaggi:

1. Clonare il repository del progetto da GitHub
2. Nella directory `robotsmart26/yamls` eseguire il comando `'docker compose -f unibobasic26.yaml up'`
3. Aprire un browser e andare su <http://localhost:8090/>
4. Eseguire all'interno della directory del progetto "robotsmart26" il comando `./gradlew run`
5. Eseguire all'interno della directory del progetto "prototype" il comando `./gradlew run`

7. Pagina di sintesi

7.1. Architettura finale del prototipo

L'architettura finale del prototipo sviluppato durante questo Sprint è riportata nella figura seguente, che costituirà il punto di partenza per lo Sprint 2.

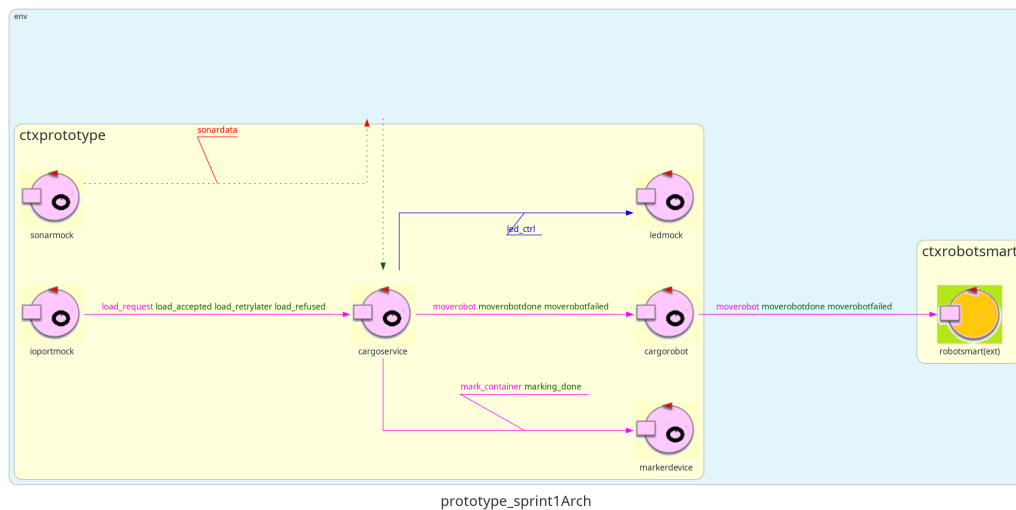


Figure 2: Architettura definita nello Sprint1.

7.2. Ripartizione del lavoro

Durante lo Sprint 1 il gruppo ha lavorato in modo collaborativo, condividendo le principali decisioni progettuali e distribuendo in maniera uniforme le attività di sviluppo.

Le principali aree di lavoro sono state:

- analisi dei requisiti e progettazione dell'architettura;
- implementazione del `CargoService`, della Hold e degli attori mock;
- integrazione con `RobotSmart` per la movimentazione del robot;
- definizione del testplan e verifica del corretto funzionamento del prototipo;
- redazione della documentazione tecnica.

7.3. Goal dello Sprint 2

L'obiettivo dello Sprint 2 è evolvere il prototipo sviluppato sostituendo progressivamente i componenti simulati con le rispettive implementazioni reali:

- IOPort come Web GUI
- sonar e LED collegati al PicoW;

Si stima una durata di circa **30 ore** complessive di lavoro (≈ 10 ore per ciascun componente del gruppo).

8. Team di lavoro

NOME E COGNOME	MATRICOLA
Davide Chirichella	0001222371
Gabriele Doti	0001245897
Daniele Maccagnan	0001247340